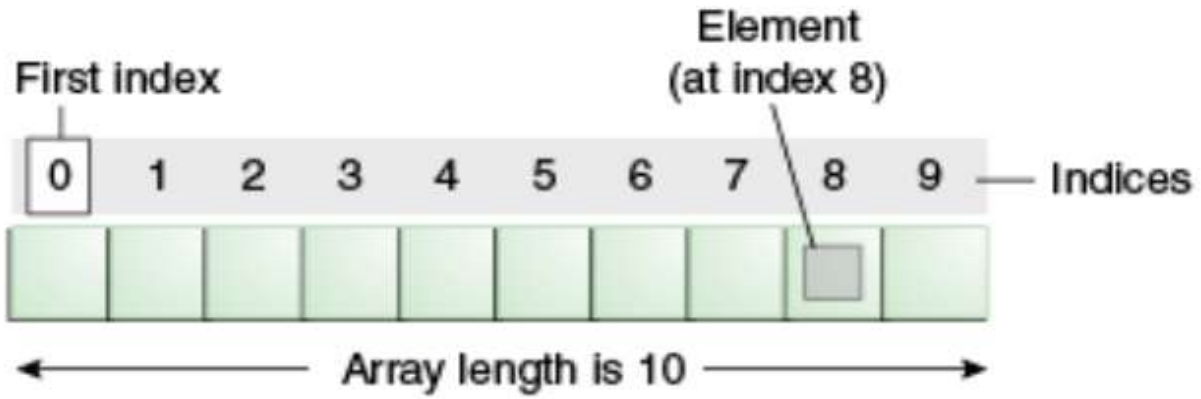
MODULE 3
ARRAYS, STRINGS AND INHERITANCE

- Java Language: Arrays:
  - 1D Arrays, Multidimensional Arrays,
  - Irregular Arrays, Array References, Using the Length Member.
  - Arrays class of util package, Array Lists, Vector class;
- Strings:
  - String class, constructors, length(), string literals,
  - concatenation, toString(), Character extraction,string comparison, s
  - earching strings, modifying, data conversion, changing the case, joining, split().
  - StringBuffer class: constructors, length(), capacity(), ensureCapacity(), setLength(), charAt(),setCharAt(),getChars(), append(), insert(), reverse(), delete(), deleteCharAt(), replace().
- Inheritance:
  - Basics, Member Access and Inheritance,
  - Constructors and Inheritance, Multilevel Hierarchy,
  - Constructor execution hierarchy, Superclass References and Subclass Objects, Method Overriding, Abstract Classes, Java Language: Using Super, Using final.

Arrays

- ✓ array is an object of a dynamically generated class
- ✓ Java array inherits the Object class, and implements the Serializable as well as Cloneable interfaces.
- ✓ We can store primitive values or objects in an array in Java.
- ✓ Like C/C++, we can also create single dimentional or multidimentional arrays in Java.



Arrays

Advantages

Code Optimization: It makes the code optimized, we can retrieve or sort the data efficiently.

Random access: We can get any data located at an index position.

Disadvantages

Size Limit: We can store only the fixed size of elements in the array. It doesn't grow its size at runtime. To solve this problem, collection framework is used in Java which grows automatically.

There are two types of array.

- ✓ Single Dimensional Array
- ✓ Multidimensional Array

Single Dimensional Array

Syntax to Declare an Array in Java

dataType[ ] array_name; (or)
datatype [ ]array_name; (or)
dataType array_name[ ];

Instantiation of an Array in Java

arrayRefVar=new datatype[size];

int a[]=new int[5]

Single Dimensional Array

```
class Testarray{
public static void main(String args[]){
int a[]=new int[5];//declaration and instantiation
a[0]=10;//initialization
a[1]=20;
a[2]=70;
a[3]=40;
a[4]=50;
//traversing array
for(int i=0;i<a.length;i++)//length is the property of arr
ay
System.out.println(a[i]);
}}
```

Single Dimensional Array

```
class Testarray1{
public static void main(String args[]){
int a[]={33,3,4,5};//declaration, instantiation and initialization

//printing array
for(int i=0;i<a.length;i++)//length is the property of arr
ay
System.out.println(a[i]);
}}
```

Single Dimensional Array

```
import java.util.Scanner;
public class ArrayInputExample1   {
  public static void main(String[] args)   {
        int n;
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter the number of elements you want to store: ");
        n=sc.nextInt();   //reading the number of elements from the that we want to enter

int[] array = new int[10];   //creates an array in the memory of length 10
        System.out.println("Enter the elements of the array: ");
        for(int i=0; i<n; i++)
          {        array[i]=sc.nextInt();  //reading array elements from the user
          }
           System.out.println("Array elements are: ");   // accessing array elements using the for loop
         for (int i=0; i<n; i++)
            {  System.out.println(array[i]);  }
    }
}
```

Single Dimensional Array

Returning Array from the Method

```
class TestReturnArray{
//creating method which returns an array
    static int[] get(){
        return new int[]{10,30,50,90,60};
    }
      public static void main(String args[]){
        // calling method which returns an array
        int arr[]=get();
       //printing the values of an array
        for(int i=0;i<arr.length;i++)
          System.out.println(arr[i]);
    }
}
```

Multidimensional Array in Java

Syntax to Declare Multidimensional Array in Java

```
dataType[][] arrayRefVar; (or)
dataType [][]arrayRefVar; (or)
dataType arrayRefVar[][]; (or)
dataType []arrayRefVar[];
```

```
int[][] arr=new int[3][3];//3 row and 3 column
```

```
data_type[1st dimension][2nd dimension][]..[Nth dimension]
array_name = new data_type[size1][size2]….[sizeN];
```

```
int[][][] threeD_arr = new int[10][20][30];
```

Multidimensional Array in Java

```
class Testarray3{
public static void main(String args[]){
//declaring and initializing 2D array
int arr[][]={{1,2,3},{2,4,5},{4,4,5}};
//printing 2D array
for(int i=0;i<3;i++){
 for(int j=0;j<3;j++){
   System.out.print(arr[i][j]+" ");
 }
 System.out.println();
}
}}
```

Multidimensional Array in Java

```
class Testarray4 {
    public static void main(String[] args)
    {

        int[][][] arr = { { { 1, 2 }, { 3, 4 } }, { { 5, 6 }, { 7, 8 } } };

        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                for (int z = 0; z < 2; z++)
                    System.out.println("arr[" + i
                                       + "]["
                                       + j + "]["
                                       + z + "] = "
                                       + arr[i][j][z]);
    }
}
```

Multidimensional Array in Java

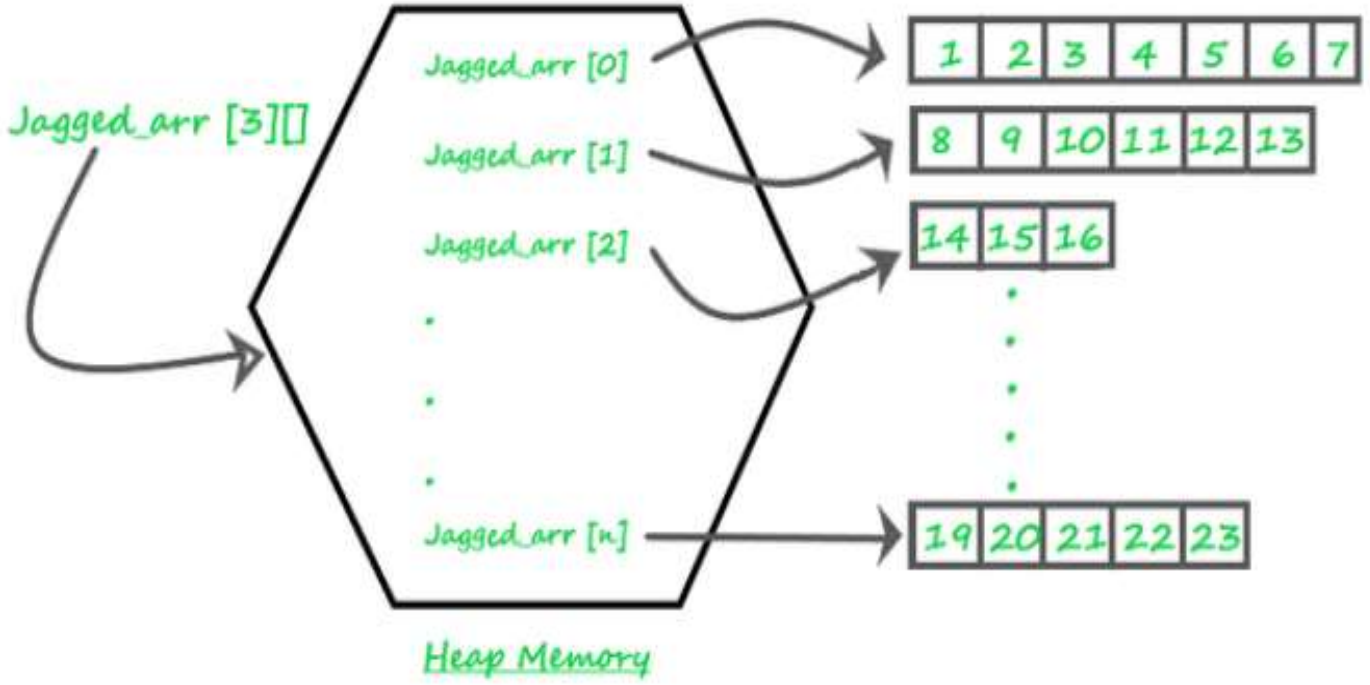
```
import java.io.*;
class GFG {
    public static void main(String[] args)
    {

        int[][][] arr = { { { 1, 2 }, { 3, 4 } },
                          { { 5, 6 }, { 7, 8 } } };

        for (int i = 0; i < 2; i++) {

            for (int j = 0; j < 2; j++) {

                for (int k = 0; k < 2; k++) {

                    System.out.print(arr[i][j][k] + " ");
                }

                System.out.println();
            }
            System.out.println();
        }
    }
}
```

Output

```
1 2
3 4

5 6
7 8
```

Irregular Arrays

A jagged array is an array of arrays such that member arrays can be of different sizes, i.e., we can create a 2-D array but with a variable number of columns in each row. These types of arrays are also known as Jagged arrays.



Jagged_array

jagged array

```
class Main {
    public static void main(String[] args)
    {
        // Declaring 2-D array with 2 rows
        int arr[][] = new int[2][];  // Making the array Jagged

        // First row has 3 columns
        arr[0] = new int[3];

        // Second row has 2 columns
        arr[1] = new int[2];

        // Initializing array
        int count = 0;
        for (int i = 0; i < arr.length; i++)
            for (int j = 0; j < arr[i].length; j++)
                arr[i][j] = count++;

        // Displaying the values of 2D Jagged array
        System.out.println("Contents of 2D Jagged Array");
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++)
                System.out.print(arr[i][j] + " ");
            System.out.println();
        }
    }
}
```

Length Member

The Java String class length() method finds the length of a string. The length of the Java string is the same as the Unicode code units of the string.

Signature

The signature of the string length() method is given below:

```
public int length()
```

Internal implementation

```
public int length() {
    return value.length;
}
```

Length Member

```
public class LengthExample{
public static void main(String args[]){
String s1="java";
String s2="python";
System.out.println("string length is: "+s1.length()); //4 is the length of pytho
n string System.out.println("string length is: "+s2.length());//6 is the length
of python string
}}
```

Arrays class of util package

Arrays class is a part of the Java collection framework in the java.util package. It only consists of static methods and methods of the object class. Using Arrays, we can create, compare, sort, search, stream, and transform arrays.

Arrays class help us to perform useful tasks on arrays conveniently without the use of loops.

Methods in Java Array Class

The static methods of the Java Array class could be used to perform operations like:

Filling the elements
Sorting the elements
Searching for the elements
Converting the array elements to String
Many more...

Methods in Java Array Class

The Arrays class of the java.util package contains several static methods that can be used to fill, sort, search, etc in arrays.

Methods	Action Performed
asList()	Returns a fixed-size list backed by the specified Arrays
binarySearch()	Searches for the specified element in the array with the help of the Binary Search Algorithm
binarySearch(array, fromIndex, toIndex, key, Comparator)	Searches a range of the specified array for the specified object using the Binary Search Algorithm
compare(array 1, array 2)	Compares two arrays passed as parameters lexicographically.
copyOf(originalArray, newLength)	Copies the specified array, truncating or padding with the default value (if necessary) so the copy has the specified length.

Methods in Java Array Class

Methods	Action Performed
sort(originalArray)	Sorts the complete array in ascending order.
sort(originalArray, fromIndex, endIndex)	Sorts the specified range of array in ascending order.
sort(T[] a, int fromIndex, int toIndex, Comparator< super T> c)	Sorts the specified range of the specified array of objects according to the order induced by the specified comparator.
sort(T[] a, Comparator< super T> c)	Sorts the specified array of objects according to the order induced by the specified comparator.

Methods in Java Array Class

```java
import java.util.Arrays;
import java.util.List;

class Scaler {

        public static void main(String[] args)
        {

                Integer arr[] = { 10, 20, 11, 21, 31 };

        List<Integer> arrList= Arrays.asList(arr);

        System.out.println("Integer Array as List: " + arrList);
        }
}
```

Output

```
Integer Array as List: [10, 20, 11, 21, 31]
```

Methods in Java Array Class 3

```java
import java.util.Arrays;

public class Scaler {

        public static void main(String[] args)
        {
                int Arr[] = { 10, 20, 11, 21, 31 };

                Arrays.sort(Arr);

                int Key = 31;

                System.out.println(
                        Key + " found at index = "
                        + Arrays.binarySearch(Arr, Key));
        }
}
```

Output

```
31 found at index = 4
```

Array Lists

Java ArrayList class uses a dynamic array for storing the elements. It is like an array, but there is no size limit. We can add or remove elements anytime.
it is found in the java.util package.

```
ArrayList<int> al = ArrayList<int>(); // does not work

ArrayList<Integer> al = new ArrayList<Integer>(); // works fine
```

Array Lists

Constructor	Description
ArrayList()	It is used to build an empty array list.
ArrayList(Collection<? extends E> c)	It is used to build an array list that is initialized with the elements of the collection c.
ArrayList(int capacity)	It is used to build an array list that has the specified initial capacity.

Method	Description
void add(int index, E element)	It is used to insert the specified element at the specified position in a list.
boolean add(E e)	It is used to append the specified element at the end of a list.
boolean addAll(Collection<? extends E> c)	It is used to append all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator.
boolean addAll(int index, Collection<? extends E> c)	It is used to append all the elements in the specified collection, starting at the specified position of the list.

Array Lists

```
import java.util.*;
 public class ArrayListExample1{
 public static void main(String args[]){
   ArrayList<String> list=new ArrayList<String>();//Creating arr
aylist
        list.add("Mango");//Adding object in arraylist
        list.add("Apple");
        list.add("Banana");
        list.add("Grapes");
        //Printing the arraylist object
        System.out.println(list);
 }
}
```

Output:

```
[Mango, Apple, Banana, Grapes]
```

Array Lists

```
import java.util.*;
class SortArrayList{
 public static void main(String args[]){
  //Creating a list of fruits
  List<String> list1=new ArrayList<String>();
  list1.add("Mango");
  list1.add("Apple");
  list1.add("Banana");
  list1.add("Grapes");
  //Sorting the list
  Collections.sort(list1);
   //Traversing list through the for-each loop
  for(String fruit:list1)
    System.out.println(fruit);
```

The java.util package provides a utility class Collections, which has the static method sort(). Using the Collections.sort() method, we can easily sort the ArrayList.

Array Lists

```
System.out.println("Sorting numbers...");
 //Creating a list of numbers
 List<Integer> list2=new ArrayList<Integer>();
 list2.add(21);
 list2.add(11);
 list2.add(51);
 list2.add(1);
 //Sorting the list
 Collections.sort(list2);
  //Traversing list through the for-each loop
 for(Integer number:list2)
   System.out.println(number);
 }
}
```

Output:

```
Apple
Banana
Grapes
Mango
Sorting numbers...
1
11
21
51
```

Vector class

Vector is like the dynamic array which can grow or shrink its size.
we can store n-number of elements in it as there is no size limit.
It is found in the java.util package and implements the List interface, so we can use all the methods of List interface here.

Java Vector class Declaration

```
public class Vector<E>
extends Object<E>
implements List<E>, Cloneable, Serializable
```

Vector class

Java Vector Constructors

Vector class supports four types of constructors. These are given below:

SN	Constructor	Description
1)	vector()	It constructs an empty vector with the default size as 10.
2)	vector(int initialCapacity)	It constructs an empty vector with the specified initial capacity and with its capacity increment equal to zero.
3)	vector(int initialCapacity, int capacityIncrement)	It constructs an empty vector with the specified initial capacity and capacity increment.
4)	Vector(Collection<? extends E> c)	It constructs a vector that contains the elements of a collection c.

Vector class

Java Vector Methods

The following are the list of Vector class methods:

SN	Method	Description
1)	add()	It is used to append the specified element in the given vector.
2)	addAll()	It is used to append all of the elements in the specified collection to the end of this Vector.
3)	addElement()	It is used to append the specified component to the end of this vector. It increases the vector size by one.
4)	capacity()	It is used to get the current capacity of this vector.
5)	clear()	It is used to delete all of the elements from this vector.
6)	clone()	It returns a clone of this vector.
7)	contains()	It returns true if the vector contains the specified element.
8)	containsAll()	It returns true if the vector contains all of the elements in the specified collection.
9)	copyInto()	It is used to copy the components of the vector into the specified array.
10)	elementAt()	It is used to get the component at the specified index.

Vector class

```
import java.util.*;
public class VectorExample {
      public static void main(String args[]) {
        //Create a vector
        Vector<String> vec = new Vector<String>();
        //Adding elements using add() method of List
        vec.add("Tiger");
        vec.add("Lion");
        vec.add("Dog");
        vec.add("Elephant");
        //Adding elements using addElement() method of Vector
        vec.addElement("Rat");
        vec.addElement("Cat");
        vec.addElement("Deer");

        System.out.println("Elements are: "+vec);
      }
}
```

Vector class

```
import java.util.*;
public class VectorExample2 {
	public static void main(String args[]) {
	 //Create an empty Vector
	 Vector<Integer> in = new Vector<>();
	 //Add elements in the vector
	 in.add(100);
	 in.add(200);
	 in.add(300);
	 in.add(200);
	 in.add(400);
	 in.add(500);
	 in.add(600);
	 in.add(700);
	 //Display the vector elements
	 System.out.println("Values in vector: " +in);
	 //use remove() method to delete the first occurence of an element
	 System.out.println("Remove first occourence of element 200: "+in.remove((Intege
r)200));
```

Vector class

```
//Display the vector elements after remove() method
        System.out.println("Values in vector: " +in);
        //Remove the element at index 4
        System.out.println("Remove element at index 4: " +in.remove(4));
        System.out.println("New Value list in vector: " +in);
        //Remove an element
        in.removeElementAt(5);
        //Checking vector and displays the element
        System.out.println("Vector element after removal: " +in);
        //Get the hashcode for this vector
        System.out.println("Hash code of this vector = "+in.hashCode());
        //Get the element at specified index
        System.out.println("Element at index 1 is = "+in.get(1));
  }
}
```

Vector class

Output:

```
Values in vector: [100, 200, 300, 200, 400, 500, 600, 700]
Remove first occourence of element 200: true
Values in vector: [100, 300, 200, 400, 500, 600, 700]
Remove element at index 4: 500
New Value list in vector: [100, 300, 200, 400, 600, 700]
Vector element after removal: [100, 300, 200, 400, 600]
Hash code of this vector = 130123751
Element at index 1 is = 300
```

Strings

string is basically an object that represents sequence of char values. An array of characters works same as Java string.

```
char[] ch={'j','a','v','a'};
String s=new String(ch);
```

is same as:

```
String s="java";
```

Java String literal is created by using double quotes.
For Example:

```
String s="welcome";
```

Strings

```
public class StringExample{
public static void main(String args[]){
String s1="java";//creating string by Java string literal
char ch[]={'s','t','r','i','n','g','s'};
String s2=new String(ch);//converting char array to string
String s3=new String("example");//creating Java string by new k
eyword
System.out.println(s1);
System.out.println(s2);
System.out.println(s3);
}}
```

Output:

```
java
strings
example
```

Strings

The java.lang.String class provides many useful methods to perform operations on sequence of char values.

No.	Method	Description
1	char charAt(int index)	It returns char value for the particular index
2	int length()	It returns string length
3	static String format(String format, Object... args)	It returns a formatted string.
4	static String format(Locale l, String format, Object... args)	It returns formatted string with given locale.
5	String substring(int beginIndex)	It returns substring for given begin index.
6	String substring(int beginIndex, int endIndex)	It returns substring for given begin index and end index.
7	boolean contains(CharSequence s)	It returns true or false after matching the sequence of char value.
8	static String join(CharSequence delimiter, CharSequence... elements)	It returns a joined string.
9	static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)	It returns a joined string.
10	boolean equals(Object another)	It checks the equality of string with the given object.
11	boolean isEmpty()	It checks if string is empty.

constructors

The most commonly used constructors of the String class are as follows:
String()
String(String str)
String(char chars[ ])
String(char chars[ ], int startIndex, int count)
String(byte byteArr[ ])
String(byte byteArr[ ], int startIndex, int count)

1. String(): To create an empty String, we will call the default constructor. The general syntax to create an empty string in java program is as follows:

```
String s = new String();
```

It will create a string object in the heap area with no value.

2. String(String str): It will create a string object in the heap area and stores the given value in it. The general syntax to construct a string object with specified string str is as follows:

```
String st = new String(String str);

For example:
    String s2 = new String("Hello Java");
```

constructors

4. String(char chars[ ], int startIndex, int count): This constructor creates and initializes a string object with a subrange of a character array.

The argument startIndex specifies the index at which the subrange begins and count specifies the number of characters to be copied.

The general syntax to create a string object with the specified subrange of character array is as follows:

```
String str = new String(char chars[ ], int startIndex, int count);
For example:
char chars[ ] = { 'w', 'i', 'n', 'd', 'o', 'w', 's'  };
String str = new String(chars, 2, 3);
```

The object str contains the address of the value "ndo" stored in the heap area because the starting index is 2 and the total number of characters to be copied is 3.

String length()

```
public class LengthExample{
public static void main(String args[]){
String s1="java";
String s2="python";
System.out.println("string length is: "+s1.length());//4 is the length of javatpoint string
System.out.println("string length is: "+s2.length());//6 is the length of python strin
g
}}
```

String concatenation

String concatenation forms a new String that is the combination of multiple strings. There are two ways to concatenate strings in Java:

1. By + (String concatenation) operator
2. By concat() method

```
class TestStringConcatenation1{
 public static void main(String args[]){
   String s="Sachin"+" Tendulkar";
   System.out.println(s);//Sachin Tendulkar
 }
}
```

```
class TestStringConcatenation3{
 public static void main(String args[]){
   String s1="Sachin ";
   String s2="Tendulkar";
   String s3=s1.concat(s2);
   System.out.println(s3);//Sachin Tendulkar
  }
}
```

String concatenation

String concatenation using StringBuilder class

```
public class StrBuilder
{
    /* Driver Code */
    public static void main(String args[])
    {
        StringBuilder s1 = new StringBuilder("Hello");    //Strin
g 1
        StringBuilder s2 = new StringBuilder(" World");    //Stri
ng 2          StringBuilder s = s1.append(s2);    //String 3 to stor
e the result
        System.out.println(s.toString());  //Displays result
    }
}
```

String concatenation

String concatenation using format() method

```
public class StrFormat
{
    /* Driver Code */
    public static void main(String args[])
    {
        String s1 = new String("Hello");    //String 1
        String s2 = new String(" World");    //String 2
        String s = String.format("%s%s",s1,s2);    //String 3 to store th
e result          System.out.println(s.toString());  //Displays result
    }
}
```

toString()

toString() method of Java Integer class is used to get a String object representing the value of the Number Object.

3 different types of Java toString() method

These are:
1.Java Integer toString() Method
2.Java Integer toString(int i) Method
3.Java Integer toString(int i, int radix) Method

Method	Returns
toString()	It returns a string representation of the value of this integer object in base 10.
toString(int i)	It returns a string representation of the int type argument in base 10.
toString(int i, int radix)	It returns a string representation of the int type argument in the specified radix.

Character extraction

The Java String class getChars() method copies the content of this string into a specified char array. There are four arguments passed in the getChars() method

Signature

```
public void getChars(int srcBeginIndex, int srcEndIndex, char[] destination, int dstBeginIndex)
```

Parameters

int srcBeginIndex: The index from where copying of characters is started.

int srcEndIndex: The index which is next to the last character that is getting copied.

Char[] destination: The char array where characters from the string that invokes the getChars() method is getting copied.

int dstEndIndex: It shows the position in the destination array from where the characters from the string will be pushed.

Character extraction

```
public class StringGetCharsExample{
public static void main(String args[]){
 String str = new String("hello javatpoint how r u");
      char[] ch = new char[10];
      try{
         str.getChars(6, 16, ch, 0);
         System.out.println(ch);
      }catch(Exception ex){System.out.println(ex);}
}}
```

Output:

```
javatpoint
```

Character extraction

charAt() Method

```
public class CharAtExample{
public static void main(String args[]){
String name="javatpoint";
char ch=name.charAt(4);//returns the char value at the 4th ind
ex
System.out.println(ch);
}}
```

Character extraction

example where we are accessing all the elements present at odd index.

```
public class CharAtExample4 {
    public static void main(String[] args) {
        String str = "Welcome to Javatpoint portal";
        for (int i=0; i<=str.length()-1; i++) {
            if(i%2!=0) {
                System.out.println("Char at "+i+" place "+str.charAt
(i));
            }
        }
    }
}
```

Character extraction

```
public class TestSubstring{
 public static void main(String args[]){
 String s="SachinTendulkar";
 System.out.println("Original String: " + s);
 System.out.println("Substring starting from index 6: " +s.substring(6));//Tendulk
ar
 System.out.println("Substring starting from index 0 to 6: "+s.substring(0,6)); //S
achin
 }
}
```

String Comparison

It is used in authentication (by equals() method), sorting (by compareTo() method), reference matching (by == operator) etc.

There are three ways to compare String in Java:
1. By Using equals() Method
2. By Using == Operator
3. By compareTo() Method

```
class Teststringcomparison1{
 public static void main(String args[]){
   String s1="Sachin";
   String s2="Sachin";
   String s3=new String("Sachin");
   String s4="Saurav";
   System.out.println(s1.equals(s2));//true
   System.out.println(s1.equals(s3));//true
   System.out.println(s1.equals(s4));//false
 }
}
```

String Comparison

By Using == operator

```
class Teststringcomparison3{
 public static void main(String args[]){
   String s1="Sachin";
   String s2="Sachin";
   String s3=new String("Sachin");
   System.out.println(s1==s2);//true (because both refer to same instance)
   System.out.println(s1==s3);//false(because s3 refers to instance created in nonpool)
 }
}
```

compareTo() method

The String class compareTo() method compares values lexicographically and returns an integer value that describes if first string is less than, equal to or greater than second string.

Suppose s1 and s2 are two String objects. If:

- **s1 == s2** : The method returns 0.
- **s1 > s2** : The method returns a positive value.
- **s1 < s2** : The method returns a negative value.

```
class Teststringcomparison4{
 public static void main(String args[]){
   String s1="Sachin";
   String s2="Sachin";
   String s3="Ratan";
   System.out.println(s1.compareTo(s2));//0
   System.out.println(s1.compareTo(s3));//1(because s1>s3)
   System.out.println(s3.compareTo(s1));//-1(because s3 < s1 )
 }
}
```

Output:

```
0
1
-1
```

Searching Strings

The **Java String class indexOf()** method returns the position of the first occurrence of the specified character or string in a specified string

Signature

There are four overloaded indexOf() method in Java. The signature of indexOf() methods are given below:

No.	Method	Description
1	int indexOf(int ch)	It returns the index position for the given char value
2	int indexOf(int ch, int fromIndex)	It returns the index position for the given char value and from index
3	int indexOf(String substring)	It returns the index position for the given substring
4	int indexOf(String substring, int fromIndex)	It returns the index position for the given substring and from index

Searching Strings

```
public class IndexOfExample{
public static void main(String args[]){
String s1="this is index of example";
//passing substring
int index1=s1.indexOf("is");//returns the index of is substring
int index2=s1.indexOf("index");//returns the index of index substring
System.out.println(index1+"  "+index2);//2 8

//passing substring with from index
int index3=s1.indexOf("is",4);//returns the index of is substring after 4th in
dex
System.out.println(index3);//5 i.e. the index of another is

//passing char value
int index4=s1.indexOf('s');//returns the index of s char value
System.out.println(index4);//3
}}
```

Output:

```
2  8
5
3
```

Data Conversion

```
public class WideningTypeCastingExample
{
public static void main(String[] args)
{
int x = 7;
//automatically converts the integer type into long type
long y = x;
//automatically converts the long type into float type
float z = y;
System.out.println("Before conversion, int value "+x);
System.out.println("After conversion, long value "+y);
System.out.println("After conversion, float value "+z);
}
}
```

Output

```
Before conversion, the value is: 7
After conversion, the long value is: 7
After conversion, the float value is: 7.0
```

Data Conversion

Narrowing Type Casting

Converting a higher data type into a lower one is called **narrowing** type casting.

```
public class NarrowingTypeCastingExample
{
public static void main(String args[])
{
double d = 166.66;
//converting double data type into long data type
long l = (long)d;
//converting long data type into int data type
int i = (int)l;
System.out.println("Before conversion: "+d);
//fractional part lost
System.out.println("After conversion into long type: "+l);
//fractional part lost
System.out.println("After conversion into int type: "+i);
}
}
```

Output

```
Before conversion: 166.66
After conversion into long type: 166
After conversion into int type: 166
```

Changing the Case

String toUpperCase()

The **java string toUpperCase()** method returns the string in uppercase letter. In other words, it converts all characters of the string into upper case letter.

```
public class StringUpperExample{
public static void main(String args[]){
String s1="hello string";
String s1upper=s1.toUpperCase();
System.out.println(s1upper);
}}
```

Output:

```
HELLO STRING
```

Changing the Case

String toLowerCase()

The **java string toLowerCase()** method returns the string in lowercase letter. In other words, it converts all characters of the string into lower case letter.

```
public class StringLowerExample{
public static void main(String args[]){
String s1="JAVA HELLO stRIng";
String s1lower=s1.toLowerCase();
System.out.println(s1lower);
}}
```

Joining

Java String join()

The **Java String class join()** method returns a string joined with a given delimiter.

There are two types of join() methods in the Java String class.

Signature

The signature or syntax of the join() method is given below:

```
public static String join(CharSequence delimiter, CharSequence... elements)
and
public static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)
```

Joining

```
public class StringJoinExample2 {
    public static void main(String[] args) {
        String date = String.join("/","25","06","2018");
        System.out.print(date);
        String time = String.join(":", "12","10","10");
        System.out.println(" "+time);
    }
}
```

Output:

```
25/06/2018 12:10:10
```

split()

The **java string split()** method splits this string against given regular expression and returns a char array

Signature

There are two signature for split() method in java string.

```
public String split(String regex)
and,
public String split(String regex, int limit)
```

split()

```
public class SplitExample{
public static void main(String args[]){
String s1="java string split method by javatpoint";
String[] words=s1.split("\\s");//splits the string based on whitespace
//using java foreach loop to print elements of string array
for(String w:words){
System.out.println(w);
}
}}
```

```
java
string
split
method
by
javatpoint
```

String Buffer Class

StringBuffer class is used to create mutable (modifiable) String objects.
The StringBuffer class in Java is the same as String class except it is mutable i.e. it can be changed.

Constructor	Description
StringBuffer()	It creates an empty String buffer with the initial capacity of 16.
StringBuffer(String str)	It creates a String buffer with the specified string..
StringBuffer(int capacity)	It creates an empty String buffer with the specified capacity as length.

String Buffer Class

```
import java.io.* ;

class A {
    public static void main(String args[])
    {
        StringBuffer sb = new StringBuffer("Hello ");
        sb.append("Java"); // now original string is changed
        System.out.println(sb);
    }
}
```

```
Hello Java
```

```
import java.io.* ;

class A{
    public static void main(String args[]){
        StringBuffer sb=new StringBuffer("Hello");
        sb.replace(1,3,"Java");
        System.out.println(sb);
    }
}
```

```
HJavalo
```

String Buffer Class

Methods of StringBuffer class

Methods	Action Performed
append()	Used to add text at the end of the existing text.
length()	The length of a StringBuffer can be found by the length( ) method
capacity()	the total allocated capacity can be found by the capacity( ) method
charAt()	This method returns the char value in this sequence at the specified index.
delete()	Deletes a sequence of characters from the invoking object
deleteCharAt()	Deletes the character at the index specified by *loc*
ensureCapacity()	Ensures capacity is at least equals to the given minimum.
insert()	Inserts text at the specified index position
length()	Returns length of the string
reverse()	Reverse the characters within a StringBuffer object
replace()	Replace one set of characters with another set inside a StringBuffer object

String Buffer Class

The **setCharAt()** method of **StringBuffer class** sets the character at the position index to character which is the value passed as parameter to method. This method returns a new sequence which is identical to old sequence only difference is a new character ch is present at position index in new sequence. The index argument must be greater than or equal to 0, and less than the length of the String contained by StringBuffer object.

Syntax: `public void setCharAt(int index, char ch)`

String Buffer Class

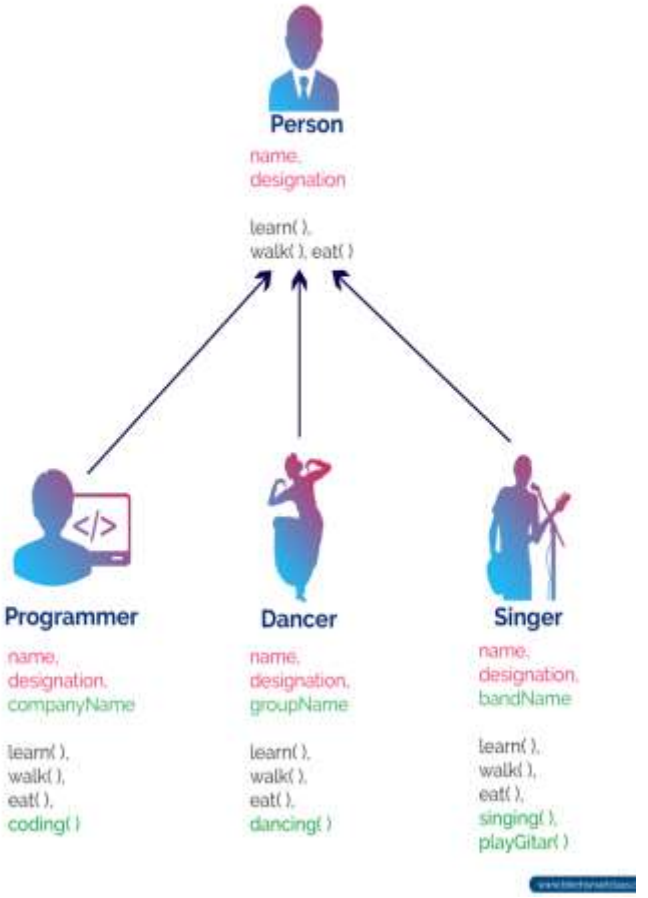
getChars() method copies the content of this string into a specified char array. There are four arguments passed in the getChars() method. The signature of the getChars() method is given below:

Syntax:

```
public void getChars(int srcBeginIndex, int srcEndIndex, char[]
 destination, int dstBeginIndex)
```

Inheritance

- Inheritance is the process of acquiring properties and behaviours from one class to another class.
- In inheritance, we derive a new class from the existing class.
- Here, the new class acquires the properties and behaviours from the existing class.
- In the inheritance concept, the class which provides properties is called as **parent class** and the class which receives the properties is called as **child** class.
- The parent class is also known as **base** class or **super** class. The child class is also known as **derived** class or **sub** class.
- In the inheritance, the properties and behaviors of base class extended to its derived class,



Inheritance

Super Class: The class whose features are inherited is known as super class(or a **base class or a parent class**).

Sub Class: The class that inherits the other class is known a sub class(or a **derived class, extended class, or child** class). The subclass can add its own fields and methods in addition to the superclass fields and methods.

Reusability: Inheritance supports the concept of “reusability”- new class (sub class) is reusing the fields and methods of the existing class (super class)

Benefits of inheritance in JAVA

1. **Reusability** - facility to use public methods of base class without rewriting the same.

2. **Extensibility** - extending the base class logic as per business logic of the derived class.

3. **Data hiding** - base class can decide to keep some data private so that it cannot be altered by the derived class

4. **Overriding** -With inheritance, we will be able to override the methods of the base class so that meaningful implementation of the base class method can be designed in the derived class.

Forms of inheritance in JAVA

- The choices between inheritance and overriding, subclass and subtypes, mean that inheritance can be used in a variety of different ways and for different purposes. Many of these types of inheritance are given their own special names.

1. **Specialization** - The child class is a special case of the parent class; in other words, the child class is a subtype of the parent class.
2. **Specification** - The parent class defines behavior that is implemented in the child class but not in the parent class.
3. **Construction** - The child class makes use of the behavior provided by the parent class but is not a subtype of the parent class.
4. **Generalization** - The child class modifies or overrides some of the methods of the parent class.

Forms of inheritance in JAVA

5. **Extension**. The child class adds new functionality to the parent class, but does not change any inherited behavior.
6. **Limitation**. The child class restricts the use of some of the behavior inherited from the parent class.
7. **Variance**. The child class and parent class are variants of each other, and the class-subclass relationship is arbitrary.
8. **Combination**. The child class inherits features from more than one parent class. This is multiple inheritance.

Costs of inheritance in JAVA

1. **Execution Speed** – Inherited methods are often slower than specialized code.

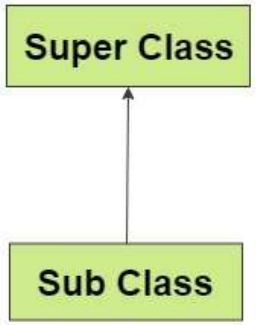
2. **Program Size** – Inheritance increases the program size when we have a huge software library to be inherited in the program.

3. **Message Passing Overhead** – As JAVA uses message passing to communicate between objects by sending and receiving messages to invoke a method within a class, inheritance will create an overhead.

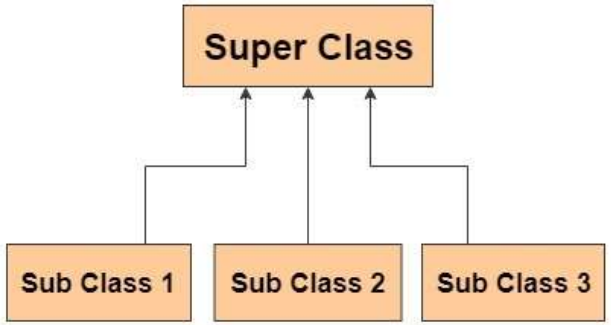
4. **Programming Complexity** – Inheritance can increase the complexity. It may replace one form of complexity with another.

A simple inheritance example in JAVA

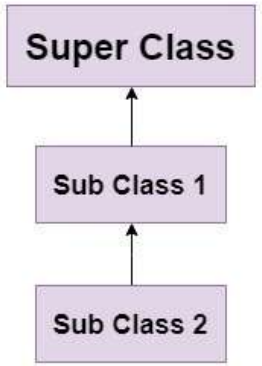
Single Inheritance



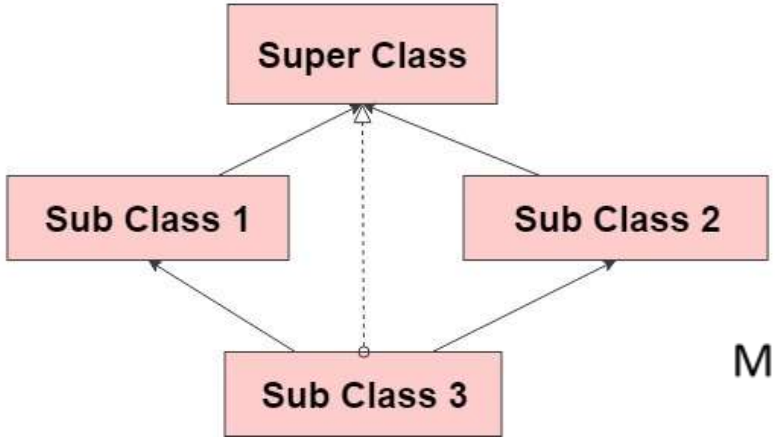
Hierarchial Inheritance



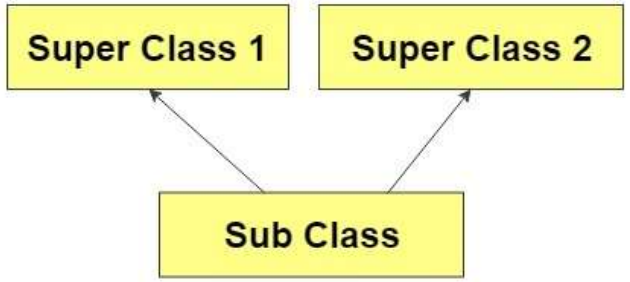
MultiLevel Inheritance



Hybrid Inheritance



Multiple Inhertance



Multiple inheritance **is supported in java through interfaces**

A simple inheritance example in JAVA

Syntax :

```
class derived-class extends base-class
{
   //methods and fields
}
```

A simple inheritance example in JAVA

```
class A {
        int i, j;
        void showij() {
                System.out.println("i and j: " + i + " " + j);
        }
}

class B extends A {
        int k;
        void showk() {
        showij();
        System.out.println("k: " + k);
        }
        void sum() {
                System.out.println("i+j+k: " + (i+j+k));
        }
}
```

A simple inheritance example in JAVA

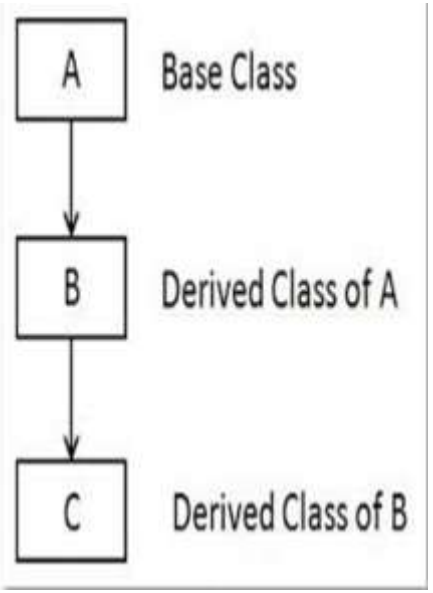
```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```

Output:

```
barking...
eating...
```

Multilevel Inheritance

- you can build hierarchies that contain as many layers of inheritance
- we can use a subclass as a superclass of another.
- For example, given three classes called A, B, and C, C can be a subclass of B, which is a subclass of A.
- When this type of situation occurs, each subclass inherits all of the traits found in all of its super classes. In this case, C inherits all aspects of B and A.
- in a class hierarchy, constructors complete their execution in order of derivation, from superclass to subclass.
- Further, since super( ) must be the first statement executed in a subclass' constructor, this order is the same whether or not super( ) is used.
- If super( ) is not used, then the default or parameter less constructor of each superclass will be executed.

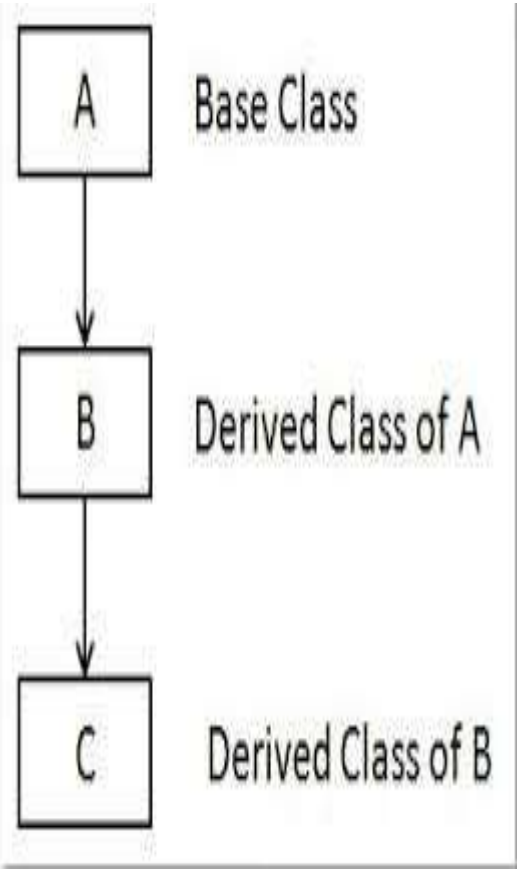


Multilevel Inheritance

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

Output:

```
weeping...
barking...
eating...
```



Multilevel Inheritance

```
public class Base {
  Base() {
   System.out.println("Base cls constructor");
   }
 public void displayBase() {
System.out.println("Base cls display"); }
}

class subclassA extends Base{
subclassA(){
System.out.println("subclassA cls constructor");
}

public void displaysubclassA()
{
System.out.println("subclassA cls display");
}
}
```

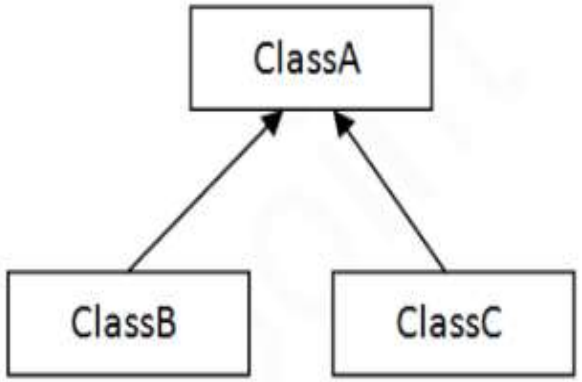
```
class subclassC extends subclassA{
        subclassC(){
        System.out.println("subclassC cls constructor");
}
public void displaysubclassC()
{ System.out.println("subclassC cls display"); }
}

public class MLInheritance {

public static void main(String[] args) {
// TODO Auto-generated method stub
subclassC c=new subclassC();
c.displayBase();
c.displaysubclassA();
c.displaysubclassC();
subclassA a= new subclassA();
a.displayBase();
a.displaysubclassA();
} }
```

Hierarchical Inheritance

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
//c.bark();//C.T.Error
}}
```

Output:

```
meowing...
eating...
```



3) Hierarchical

Uses of Super keyword in JAVA

Super is a reference variable that is used to refer to the immediate parent class objects.

Uses:

1. Super is used to invoke the immediate parent class constructor.
2. Super is used to invoke the immediate parent class method.
3. Super is used to invoke the immediate parent class variable.

Uses of Super keyword in JAVA

```
class Animal{
String color="white";
void message(){
System.out.print("\nIn base class- Animal-\n");
System.out.println("color:" +color);
}
}
class Dog extends Animal
{
String color="black";
void message()// accessing members variable using super
{
System.out.println("In sub class- Dog" );
//prints member color=black of child class- Dog
System.out.println(" sub class Color = "+ color);
//prints parent class- Animal member
color =white
System.out.println("Super class colour =
"+super.color);
}
```

```
void display()// accessing members method using super
{

message();//will invoke sub class message() method
super.message();//will invoke parent class message() method

}


}


public class TestSuper1{

public static void main(String args[]){
Dog d=new Dog();
d.display();//subclass
}

}
```

```
In sub class- Dog
sub class Color = black
Super class colour = white

In base class- Animal-
color: white
```

Uses of Super keyword in JAVA

```
class Animal{
String color="white";
void message(){
System.out.print("\nIn base class- Animal-\n");
System.out.println("color:" +color);
}
}
class Dog extends Animal
{
String color="black";
void message()// accessing members variable using super
{
System.out.println("In sub class- Dog" );
//prints member color=black of child class- Dog
System.out.println(" sub class Color = "+ color);
//prints parent class- Animal member
color =white
System.out.println("Super class colour =
"+super.color);
}
```

```
void display()// accessing members method using super
{

message();//will invoke sub class message() method
super.message();//will invoke parent class message() method

}


}


public class TestSuper1{

public static void main(String args[]){
Dog d=new Dog();
d.display();//subclass
}

}
```

```
In sub class- Dog
sub class Color = black
Super class colour = white

In base class- Animal-
color: white
```

Uses of Super keyword in JAVA

```
class superA {
    superA() {
      System.out.println("Base cls A constructor");
    }
}

class subB extends superA{
      subB() {
         super();
         System.out.println("sub cls B constructor");
      }
}

public class TestConstSuper {
    public static void main(String[] args) {
         // TODO Auto-generated method stub
         subB b=new subB();
    }
}
```

OUTPUT:
Base cls A constructor
sub cls B constructor

Access Modifiers

		public	private	protected	default
Same Package	Class	YES	YES	YES	YES
	Sub class	YES	NO	YES	YES
	Non sub class	YES	NO	YES	YES
Different Package	Sub class	YES	NO	YES	NO
	Non sub class	YES	NO	NO	NO

Method Overriding

If subclass (child class) has the same method as declared in the parent class, it is known as method overriding in Java.

Rules for Java Method Overriding

1.The method must have the same name as in the parent class
2.The method must have the same parameter as in the parent class.
3.There must be an IS-A relationship (inheritance).

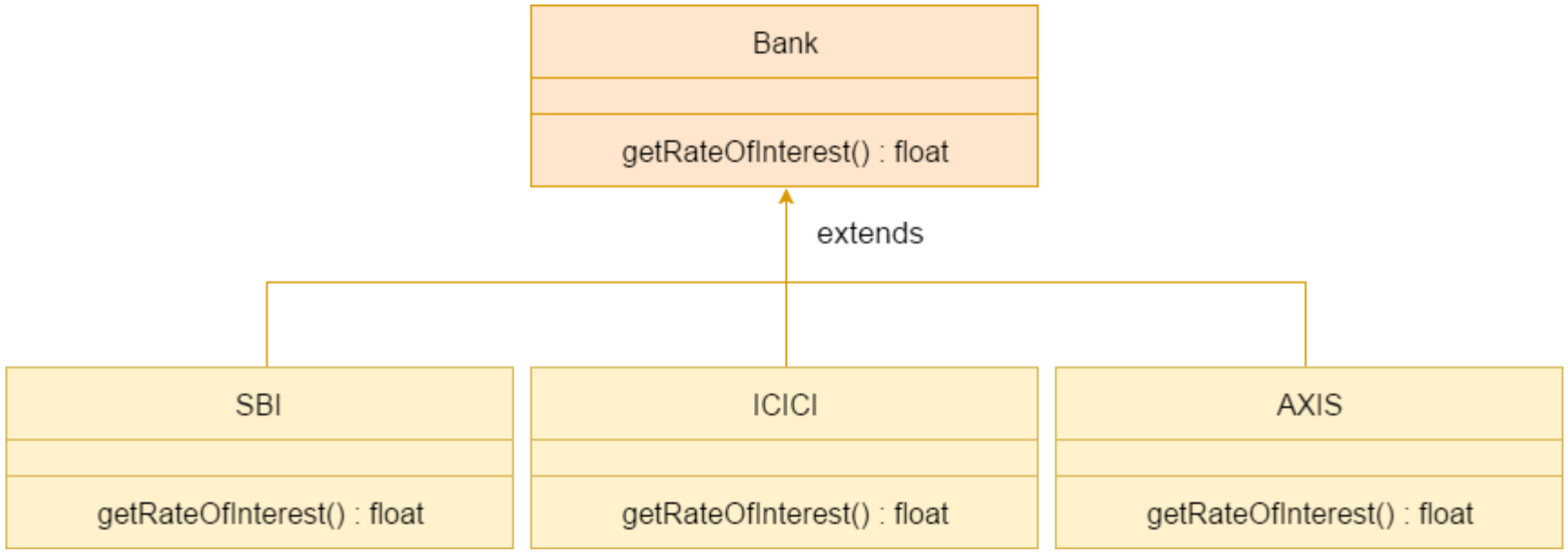
Method Overriding

```
//Java Program to illustrate the use of Java Method Overriding
//Creating a parent class.
class Vehicle{
  //defining a method
  void run(){System.out.println("Vehicle is running");}
}
//Creating a child class
class Bike2 extends Vehicle{
  //defining the same method as in the parent class
  void run(){System.out.println("Bike is running safely");}

  public static void main(String args[]){
  Bike2 obj = new Bike2();//creating object
  obj.run();//calling method
  }
}
```

Output:
Bike is running safely

Method Overriding

A real example of Java Method Overriding



Method Overriding

```
class Bank{
int getRateOfInterest(){return 0;}
}
//Creating child classes.
class SBI extends Bank{
int getRateOfInterest(){return 8;}
}

class ICICI extends Bank{
int getRateOfInterest(){return 7;}
}
class AXIS extends Bank{
int getRateOfInterest(){return 9;}
}
```

```
   //Test class to create objects and call the meth
   ods
class Test2{
public static void main(String args[]){
SBI s=new SBI();
ICICI i=new ICICI();
AXIS a=new AXIS();
System.out.println("SBI Rate of Interest: “
            +s. getRateOfInterest());
System.out.println("ICICI Rate of Interest:
 “
            +i. getRateOfInterest());
System.out.println("AXIS Rate of Interest:
 “
            +a .getRateOfInterest());
}
}
```

Output:

SBI Rate of Interest: 8
ICICI Rate of Interest: 7
AXIS Rate of Interest: 9

Abstract Classes

Abstraction is a **process of hiding the implementation details from the user, only the functionality will be provided to the user**. In other words, the user will have the information on what the object does instead of how it does it.

•Data Abstraction may also be defined as the **process of identifying only the required characteristics of an object ignoring the irrelevant details**.

Ex: 1) A car is viewed as a car rather than its individual components.

Ways to achieve Abstraction

There are two ways to achieve abstraction in java

1.Abstract class (0 to 100%)

2.Interface (100%)

Abstract Classes

- There are situations in which you will want to define a **superclass that declares the structure of a given abstraction** without providing a complete implementation of every method.
- Sometimes you will want to create a **superclass that only defines a generalized form that will be shared by all of its subclasses**, leaving it to each subclass to fill in the details.
- Such a class determines the nature of the methods that the subclasses must implement.

1. An **abstract class** is a class that is declared with **abstract keyword**.

2. An **abstract method** is a method that is **declared without an implementation**.

3. An **abstract class may or may not have all abstract methods**. Some of them can be **concrete methods**.

4. A **method defined abstract must always be redefined in the subclass**, thus making **overriding compulsory** OR either make subclass itself abstract.

Abstract Classes

5. **Any class that contains one or more abstract methods** must also be **declared with abstract keyword**.

6. There can be **no object of an abstract class**. That is, an abstract class **cannot be directly instantiated** with the new operator.

7. An **abstract class can have parametrized constructors** and default constructor is always present in an abstract class.

8. We can have an abstract class without any abstract method. This allows us to create classes that cannot be instantiated but can only be inherited.

Abstract Classes

```
abstract class Vehicle
{  //variable that is used to declare the no. of wheels in a vehicle
   private int wheels;    //Variable to define the type of motor used
  private Motor motor;
     //an abstract method that only declares, but does not define the start
  //functionality because each vehicle uses a different starting mechanism
   abstract void start();
}
public class Car extends Vehicle  { ...    }
public class Motorcycle extends Vehicle { ... }
```

```
Vehicle newVehicle = new Vehicle();    // Invalid
Vehicle car = new Car();  // valid
Vehicle mBike = new Motorcycle();  // valid

Car carObj = new Car();  // valid
Motorcycle mBikeObj = new Motorcycle();  // valid
```

Abstract Classes

```
abstract class shape {
 public int x, y;
 public abstract void printArea();
}
class Rectangle extends shape {
 public void printArea() {
 System.out.println("Area of Rectangle is " + x * y);
 }
}
class Triangle extends shape {
 public void printArea() {
 System.out.println("Area of Triangle is " + (x * y) / 2);
 }
}
```

```
class Circle extends shape {
 public void printArea() {
 System.out.println("Area of Circle is " + (float)(22 * x * x) / 7);
 }  }
public class Abstex {
 public static void main(String[] args) {
  Rectangle r = new Rectangle();
  r.x = 10;  r.y = 20;
  r.printArea();
 Triangle t = new Triangle();
  t.x = 30;  t.y = 35;
  t.printArea();
 Circle c = new Circle();
  c.x = 2;
  c.printArea();
  }
}
```

Using final with Inheritance-(Using final to Prevent Overriding) Classes

The final keyword is a non-access modifier used for classes, attributes and methods, which makes them non-changeable (impossible to inherit or override).

Different Ways to Prevent Method Overriding in Java

Methods:

1. Using a static method
2. Using private access modifier
3. Using the final keyword method

Using final with Inheritance-(Using final to Prevent Overriding) Classes

- To **disallow a method from being overridden**, specify final as a modifier at the start of its declaration.
- Methods declared as final cannot be overridden.

```
class A {
        final void meth() {
                System.out.println("This is a final method.");
        }
}

class B extends A {
        void meth() { // ERROR! Can't override.
                System.out.println("Illegal!");
        }
}
```